

Object Oriented Programming

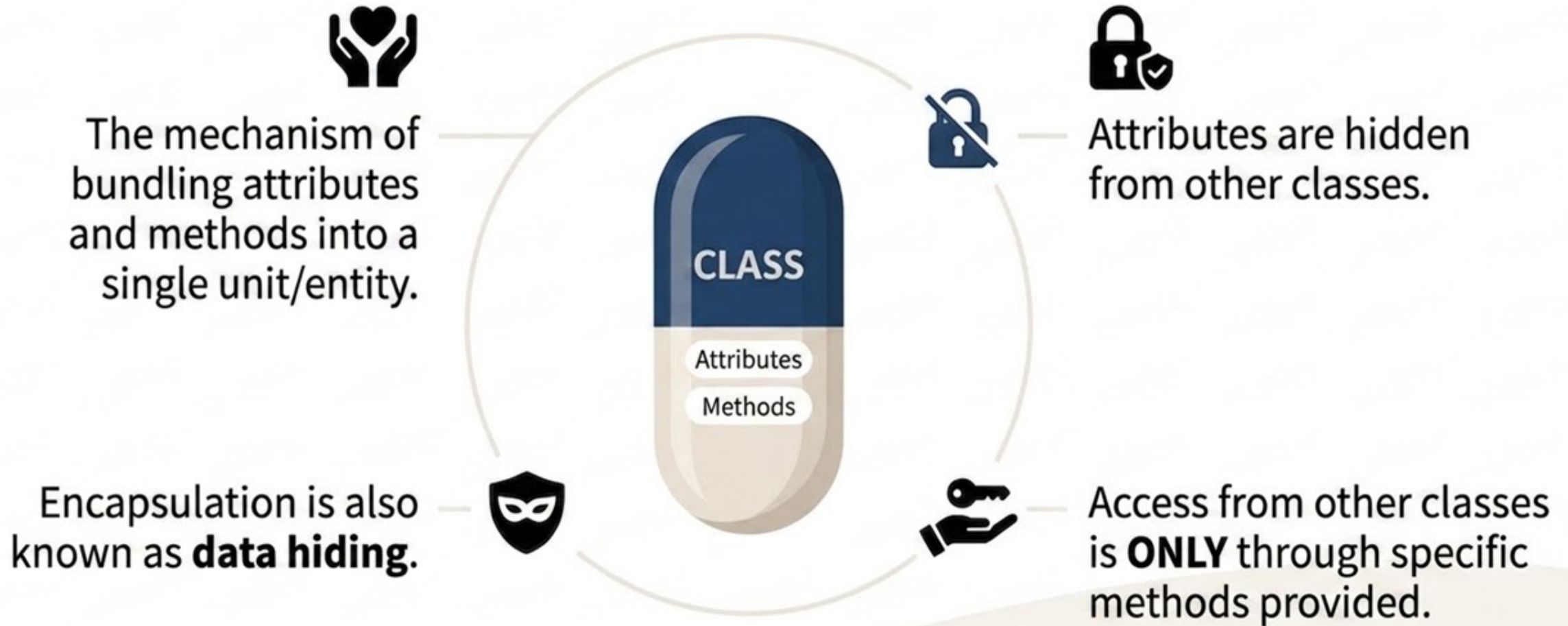
Encapsulation

OOP Team

Outline

- Access Modifier
 - Getter – Setter
 - Constructor
 - UML Notation
- 

Encapsulation



Analyzing Scenarios

Let's observe the behavior of real-world objects.

- For example, a car.
- How does a car behave?
 - Turn on the engine. Increase speed. Increase speed again. Reduce speed. Stop. Reverse slowly. Stop.
- This behavior can be executed through several mechanisms.
 - The ignition key to turn the engine on and off. The gear shift, gas pedal, and brake pedal to increase or decrease speed.
- In this case, we use the gas pedal and brake pedal to regulate the car's speed.

Analyzing Scenarios

```
public class Car
{
    public int speed = 0;
    public boolean ignitionOn = false;

    public void displayStatus()
    {
        if(ignitionOn == true)
        {
            System.out.println("Ignition ON");
        }
        else
        {
            System.out.println("Ignition OFF");
        }

        System.out.println("Speed: " + speed + "\n");
    }
}
```

```
public class TestCar
{
    public static void main(String[] args)
    {
        Car myCar = new Car();

        myCar.displayStatus();
        myCar.speed = 100;
        myCar.displayStatus();
    }
}
```

```
Ignition OFF
Speed: 0
```

```
Ignition OFF
Speed: 100
```

Analyzing Scenarios

Something is strange about the Mobil (Car) class.

- Specifically, the car's speed suddenly changes from 0 to 100.
- Even stranger, the car's ignition position is still OFF.
- How is it possible for a car to instantly accelerate from zero to 100 while the ignition key is OFF?
- In this case, access to the Car's attributes is uncontrolled.
- Yet, real-world objects always have limitations and mechanisms regarding how they can be used.

Solution

- Hiding internal attributes (speed, ignitionOn) from the user (other classes).
 - Providing special methods to access the attributes.
- 

Access Modifier

- Java provides 4 different access modifiers to control access to attributes and methods from other classes, namely:
 - *private* – the most restrictive, only accessible within the class itself.
 - *default* – can only be accessed within the same package.
 - *protected* – can be accessed in packages and sub classes.
 - *public* – at the very least, accessible from anywhere.

	Same Class Same Package	Other Class Same Package	SubClass Other Package	Other Class Other Package
public	✓	✓	✓	✓
protected	✓	✓	✓	
default	✓	✓		
private	✓			

Default Access Modifier (Package-Private)

Characteristics: This modifier does not require any keyword when declared. Attributes or methods using this access level can only be accessed by other classes within the same package.

Case Example: Continuing the banking system scenario, suppose we have an internal class named **KaryawanBank** (BankEmployee) tasked with processing customer data within the same package (**perbankan_internal**), but we do not want this data to be accessed by external packages.

```
5 package internalBanking;
6
7 /**
8  *
9  * @author dian
10 */
11 public class BankEmployee {
12     // Menggunakan default modifier (tanpa keyword)
13     String employeeId;
14     String department;
15
16     // Default constructor
17     BankEmployee(String id, String dept) {
18         this.employeeId = id;
19         this.department = dept;
20     }
21
22     // Default method
23     void processTransaction() {
24         System.out.println("Employee " + employeeId +
25                             + " is processing the transaction.");
26     }
27 }
```

Protected Access Modifier

Characteristics: Attributes or methods declared with the protected keyword can be accessed by other classes within the same package, as well as by sub-classes (derived classes) even if the sub-class resides in a different package.

Case Example: Still within the banking system, a general bank account (**AkunBank**) has basic management mechanisms. We want to create specific sub-classes like **AkunGiro** (CheckingAccount) or **AkunPremium** (PremiumAccount) that can inherit and modify certain internal parameters without opening them freely (**public**) to external objects.

```
5  package mainServices;
6
7  /**
8   *
9   * @author dian
10  */
11  public class BankAccount {
12      // Using protected modifier
13      protected String accountNumber;
14      protected double interestRate;
15
16      public BankAccount(String accountNumber, double interestRate) {
17          this.accountNumber = accountNumber;
18          this.interestRate = interestRate;
19      }
20
21      // Protected method for internal adjustment
22      protected void adjustInterest(double newInterest) {
23          this.interestRate = newInterest;
24          System.out.println("Interest rate for account " + accountNumb
25                          " adjusted to: " + interestRate + "%");
26      }
27  }
```

Application on Sub-Classes (Different Package)

```
package specialServices;

import mainServices.BankAccount;

/**
 *
 * @author dian
 */
public class PremiumAccount extends BankAccount {
    private String specialBenefit;

    public PremiumAccount(String accountNumber, double interestRate, String benefit) {
        // Calling the super-class constructor
        super(accountNumber, interestRate);
        this.specialBenefit = benefit;
    }

    public void giveInterestBonus() {
        // The 'interestRate' attribute from BankAccount
        // can be accessed directly because it is protected
        double bonusInterest = interestRate + 1.5;

        // The 'adjustInterest' method can be called directly by the derived class
        adjustInterest(bonusInterest);
        System.out.println("Additional benefit: " + specialBenefit);
    }
}
```

UML Notation for Access Modifiers

- The UML notations in class diagrams for access modifiers are:
 - Minus sign (-) for private
 - Hash sign (#) for protected
 - Plus sign (+) for public
 - Tilde sign (~) or no sign for default

SpaceShuttle
+ color: String - speed: int - orbitHeight: int # fuelType: String # engineType: String nickName: String
+ start(): void + climb(): void + dive(): void + getSpeed(): int + getOrbitHeight(): int

Implementing Access Modifiers

Mobil
- kecepatan: int - kontakOn: boolean
+ nyalakanMesin(): void + matikanMesin(): void + tambahKecepatan(): void + kurangiKecepatan(): void + tampilkanStatus(): void

```

public class Car
{
    // The speed and ignitionOn attributes are set to private
    private int speed = 0;
    private boolean ignitionOn = false;

    public void turnOnEngine()
    {
        ignitionOn = true;
    }

    public void turnOffEngine()
    {
        ignitionOn = false;
    }

    public void increaseSpeed()
    {
        if (ignitionOn == true)
        {
            speed += 10;
        }
        else
        {
            System.out.println("Cannot increase speed, engine is not running!\n");
        }
    }

    public void decreaseSpeed()
    {
        if (ignitionOn == true)
        {
            speed -= 10;
        }
        else
        {
            System.out.println("Cannot decrease speed, engine is not running!\n");
        }
    }

    public void displayStatus()
    {
        if (ignitionOn == true)
        {
            System.out.println("Ignition ON");
        }
        else
        {
            System.out.println("Ignition OFF");
        }

        System.out.println("Speed: " + speed + "\n");
    }
}

```

```

public class TestCar
{
    public static void main(String[] args)
    {
        Car car = new Car();

        car.displayStatus();

        car.increaseSpeed();

        car.turnOnEngine();
        car.displayStatus();

        car.increaseSpeed();
        car.displayStatus();

        car.increaseSpeed();
        car.displayStatus();

        car.increaseSpeed();
        car.displayStatus();
    }
}

```

```

Ignition OFF
Speed: 0

Cannot increase speed, engine is not running!

Ignition ON
Speed: 0

Ignition ON
Speed: 10

Ignition ON
Speed: 20

Ignition ON
Speed: 30

```

Implementing Access Modifiers

- We can now observe that the speed attribute cannot be changed arbitrarily.
- In fact, when attempting to increase speed while the ignition is still OFF, a notification will appear stating that the engine has not started.
- To reach the desired speed, it must be done gradually by calling the `tambahKecepatan()` (`increaseSpeed`) method multiple times.
- This closely resembles how we drive a car in the real world.

Getter – Setter Method

- Getter – Setter methods are methods provided to access attributes that are set to private.
 - Getters are used to read the content of an attribute, while Setters are used to set the content of an attribute.
 - Thus, we can determine whether an attribute is read-only, write-only, or both readable and writable.
- 

Analyzing Scenarios

- Suppose in a banking information system, there is a Nasabah (Customer) class.
- The Customer has attributes for account number, name, and balance, alongside deposit() and withdraw() methods.
- The customer's balance must not be changed arbitrarily; instead, it should only be modified through the deposit() or withdraw() methods.
- However, the nominal balance value is allowed to be read.

Implementing Getter – Setter Method

Nasabah
- nomorRekening: String - nama: String - saldo: int
+ setNomorRekening(norek: String): void + getNomorRekening(): String + setName(nm: String): void + getName(): String + getSaldo(): int + setor(nominal: int): void + tarik(nominal: int): void

```
public class Customer
{
    private String accountNumber;
    private String name;
    private int balance = 0;

    public void setAccountNumber(String accNo)
    {
        accountNumber = accNo;
    }

    public String getAccountNumber()
    {
        return accountNumber;
    }

    public void setName(String name)
    {
        this.name = name;
    }

    public String getName()
    {
        return name;
    }

    public int getBalance()
    {
        return balance;
    }

    public void deposit(int amount)
    {
        balance += amount;
    }

    public void withdraw(int amount)
    {
        balance -= amount;
    }
}
```

Implementing Getter – Setter Method

- If you notice the Customer class above, the accountNumber attribute has setAccountNumber() and getAccountNumber() as its getter-setter.
- The name attribute also has setName() and getName().
- However, the balance attribute ONLY has getBalance(), because as initially intended, the balance attribute must not be changed arbitrarily and must go through the deposit() and withdraw() methods.

Implementing Getter – Setter Method

```
public class TestCustomer
{
    public static void main(String[] args)
    {
        Customer anton = new Customer();

        anton.setAccountNumber("1123123");
        anton.setName("Anton Kemang");
        anton.deposit(10000);

        System.out.println(anton.getName() + "'s current balance: $" + anton.getBalance());

        anton.withdraw(5000);

        System.out.println(anton.getName() + "'s current balance: $" + anton.getBalance());
    }
}
```

```
Anton Kemang's current balance: $10000
Anton Kemang's current balance: $5000
```

Implementing Getter – Setter Method

- As seen in the experiment results above:
 - Modifying the balance is not done directly by changing the balance attribute, but rather through the deposit() and withdraw() methods.
 - Similarly, displaying the name must go through the getName() method, and displaying the balance goes through getBalance().

Constructor

- A constructor is a method that automatically executes for the first time when we create an object.
- By default, Java automatically provides a constructor for every class.
- This is proven when we create a new object from a class, which is always followed by a call to its constructor.
- For example, creating a porsche object from the Car class:

```
Car porsche = new Car();
```

- Notice the code **new Car()**, which means it is calling the constructor of the Car class.

Rules for Creating a Constructor

- A constructor must have the exact same name as the class.
- Its access modifier must be public.
- It must not have a return data type.
- An example of using a constructor is when we want to provide default values to class attributes.
- The way to declare a constructor is as follows:

```
public <ConstructorName>
{
    // do something
}
```

Constructor Example

- For example, in the Class class:
 - It has attributes color and numberOfWheels.
 - We want the color to be immediately filled with White and the number of wheels to be "4" as soon as the object is created.

```
public class Car
{
    public String color;
    public int numberOfWheels;

    public Car()
    {
        color = "White";
        numberOfWheels = 4;
    }
}
```

```
public class TestCar
{
    public static void main(String[] args)
    {
        Car car = new Car();

        System.out.println("Car color: " + car.color);
        System.out.println("Number of wheels: " + car.numberOfWheels);
    }
}
```

```
Car color: White
Number of wheels: 4
```

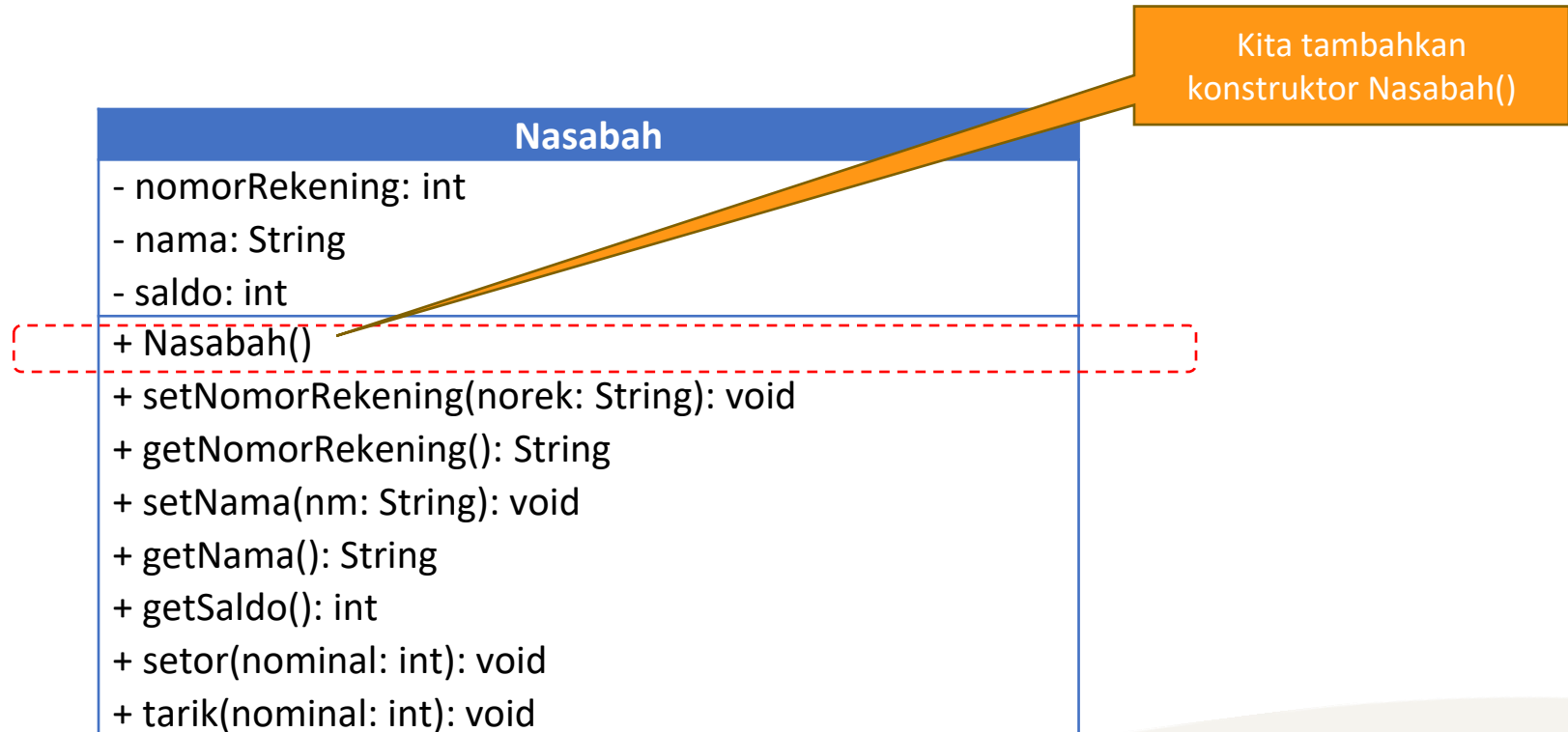
Atribut warna dan jumlahRoda otomatis terisi

Mengeset atribut warna dan jumlahRoda di dalam konstruktor

Analyzing Scenarios

- Take our previous banking system example: when an object of the Customer class is first created, the default values for the attributes should be:
 - accountNumber = "000000"
 - name = "NO NAME"
 - balance = 50000
- How can we ensure these attributes are automatically filled with their default values upon object creation?

Implementing Constructors



```

public class Customer
{
    private String accountNumber;
    private String name;
    private int balance = 0;

    // Customer class constructor
    public Customer()
    {
        accountNumber = "000000";
        name = "NO NAME";
        balance = 50000;
    }

    public void setAccountNumber(String accNo)
    {
        accountNumber = accNo;
    }

    public String getAccountNumber()
    {
        return accountNumber;
    }

    public void setName(String name)
    {
        this.name = name;
    }

    public String getName()
    {
        return name;
    }

    public int getBalance()
    {
        return balance;
    }

    public void deposit(int amount)
    {
        balance += amount;
    }

    public void withdraw(int amount)
    {
        balance -= amount;
    }
}

```

```

public class TestCustomer
{
    public static void main(String[] args)
    {
        Customer cust = new Customer();

        System.out.println("Account number: " + cust.getAccountNumber());
        System.out.println("Name: " + cust.getName());
        System.out.println("Balance: " + cust.getBalance());
    }
}

```

```

Account number: 000000
Name: NO NAME
Balance: 50000

```

Implementing Constructors

- Observe the TestCustomer class above.
- First, we create a cust object from the Customer class.
- Then, we immediately print the contents of its attributes to the screen using its getter methods: `getAccountNumber()`, `getName()`, and `getBalance()`.
- We can see that the account number attribute is automatically filled with "000000", the name attribute with "NO NAME", and the balance = 50000.

Constructor That Accepts Parameters

- Just like a regular method, a constructor can also accept parameters.
- If a constructor accepts parameters, then during object creation, those parameters must be assigned values via parameter passing.
- Example:

```
Mahasiswa mhs = new Mahasiswa("114003005", "Edna", 19, "Malang");
```

- This type of constructor is typically used when an object's creation must be accompanied by setting the initial values of several of its attributes.

Analyzing Scenarios

- For example, at the moment of creating an object from the Customer class, the name, account number, and balance attributes must be filled in immediately.
 - How do we pass these attributes directly through its constructor?
- 

Implementing Constructors with Parameters

Nasabah
- nomorRekening: int - nama: String - saldo: int
+ Nasabah(norekAwal: String, namaAwal: String, saldoAwal: int)
+ setNomorRekening(norek: String): void + getNomorRekening(): String + setName(nm: String): void + getName(): String + getSaldo(): int + setor(nominal: int): void + tarik(nominal: int): void

```

public class Customer
{
    private String accountNumber;
    private String name;
    private int balance = 0;

    // Customer class constructor that accepts parameters
    public Customer(String initialAccNo, String initialName, int initialBalance)
    {
        accountNumber = initialAccNo;
        name = initialName;
        balance = initialBalance;
    }

    public void setAccountNumber(String accNo)
    {
        accountNumber = accNo;
    }

    public String getAccountNumber()
    {
        return accountNumber;
    }

    public void setName(String name)
    {
        this.name = name;
    }

    public String getName()
    {
        return name;
    }

    public int getBalance()
    {
        return balance;
    }

    public void deposit(int amount)
    {
        balance += amount;
    }

    public void withdraw(int amount)
    {
        balance -= amount;
    }
}

```

```

public class TestCustomer
{
    public static void main(String[] args)
    {
        // Passing parameters to the Customer constructor
        Customer cust = new Customer("11556677", "Nikiloe Tesla", 900000);

        System.out.println("Account number: " + cust.getAccountNumber());
        System.out.println("Name: " + cust.getName());
        System.out.println("Balance: " + cust.getBalance());
    }
}

```

```

Account number: 11556677
Name: Nikiloe Tesla
Balance: 900000

```

Questions

- What is the main purpose of using encapsulation?
 - Provide a real-world object case example where we must apply the concept of encapsulation, including:
 - Using a private access modifier on its attributes.
 - Using specific methods to change attribute values.
 - Provide a real-world object case example where we must create a constructor within a class.
- 